LECTURE 6

DECORATOR DESIGN PATTERN

Adding Responsibilities Dynamically

Structural Pattern

Java Implementation

Composition over Inheritance



OBJECTIVE: *Learn how to add new functionality to objects at runtime without modifying their class.*

Decorator
Design Pattern

Structural Design Pattern



Learning Outcomes

By the end of this lecture students can

01 Explain why **inheritance** sometimes fails

02 Describe the **Decorator Pattern**

03 Draw the **UML**

04 **Implement** Decorator in **Java**

05 **Compare** Decorator with **Proxy**

06 Identify suitable **real-world applications**

Where Are We?

Design Patterns



- **Creational**
  - Singleton
  - Factory Method
- **Structural**
  - Facade
  - Adapter
  - Proxy
- **Decorator (Today)**

We are progressively building our understanding of Design Patterns. ***Today, we explore Decorator!***

Review Activity (5 minutes)

Which pattern would you use?

Read each scenario and decide which design pattern is the most suitable.

Scenario 1



Only one printer manager should exist.

Answer: Singleton

Scenario 2



Create different payment objects.

Answer: Factory Method

Scenario 3



Hide a complex banking subsystem.

Answer: Facade

Scenario 4



Convert USB-C device to HDMI.

Answer: Adapter

Scenario 5



Control access to premium videos.

Answer: Proxy

Problem Scenario

Imagine a coffee shop.

Customers order



Espresso



Latte



Cappuccino

Then they request

- Milk
- Sugar
- Honey
- Chocolate
- Caramel
- Vanilla
- Extra Shot



Question

Should we create a new class for every combination?

Class Explosion

If we use **inheritance**

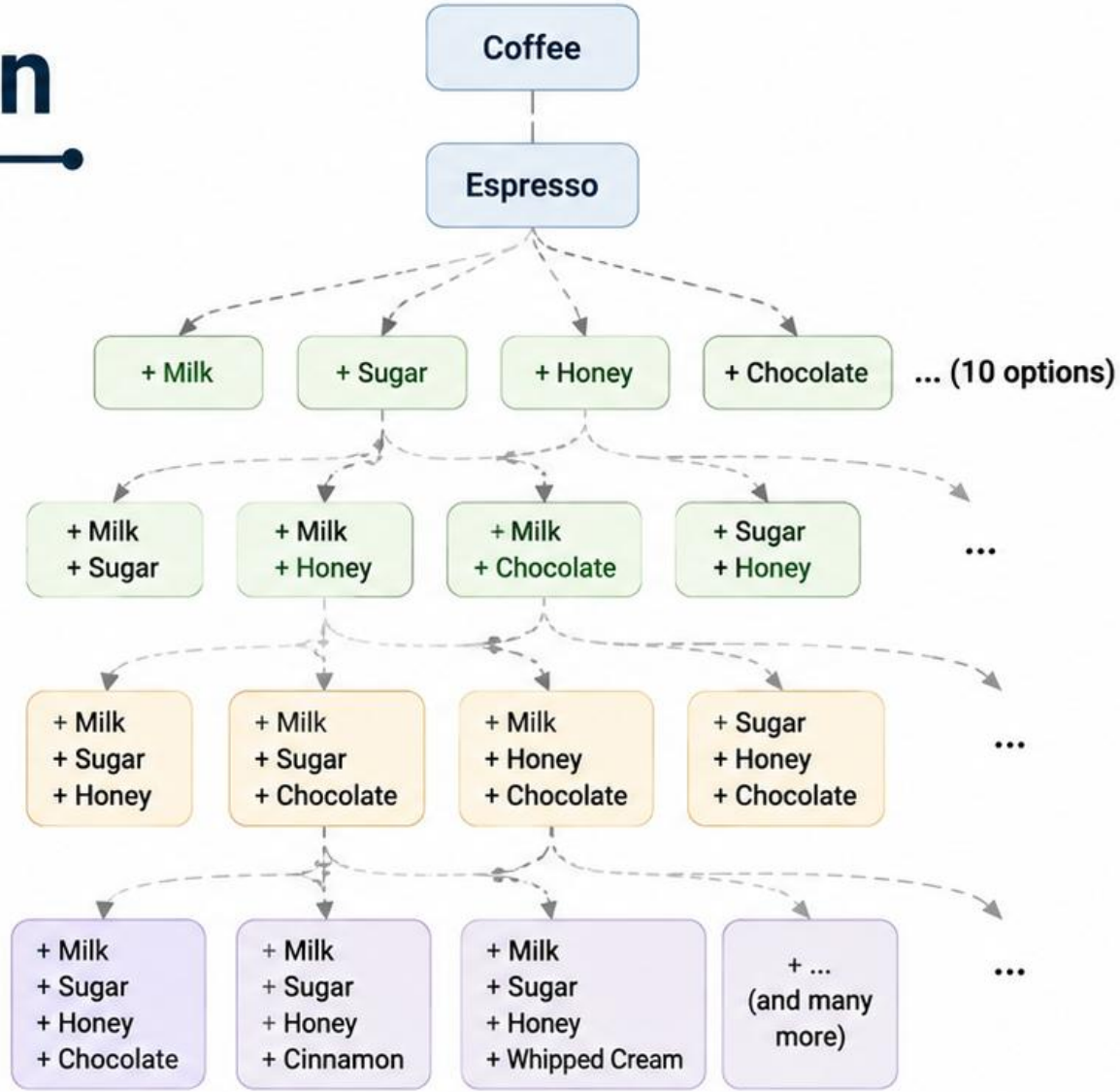
- Coffee
- **Espresso**
- **EspressoWithMilk**
- **EspressoWithSugar**
- **EspressoWithMilkSugar**
- **EspressoWithMilkSugarHoney**
- ⋮

Imagine

10 optional toppings



...



How many subclasses?

Over 1000 combinations

$$\sum_{k=0}^{10} \binom{10}{k} = 2^{10} = 1024$$

This illustrates the **"class explosion"** problem.

Inheritance leads to an ***exponential*** number of subclasses when features are optional and combinable.

We need a **better** way to add behavior **without** creating many subclasses.

Instead of creating subclasses

Wrap the object.

Instead of creating many subclasses for every combination, we **wrap** the original object with decorator objects that **add behavior**.

Each wrapper adds one responsibility.



Key Idea

Each decorator **wraps** the object and adds a **responsibility** without changing the original object.

The object is **wrapped** multiple times, and each wrapper adds **one responsibility**.

What is the Decorator Pattern?

Definition

The Decorator Pattern allows **additional responsibilities** to be attached to an object **dynamically** without changing its class.

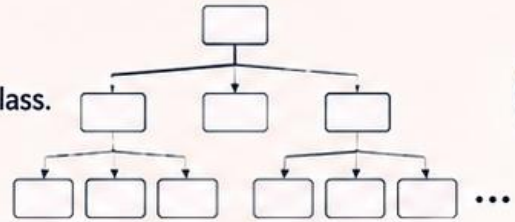


Key Ideas

- Runtime extension
- Composition over inheritance
- Open/Closed Principle
- Flexible combinations

Using Inheritance

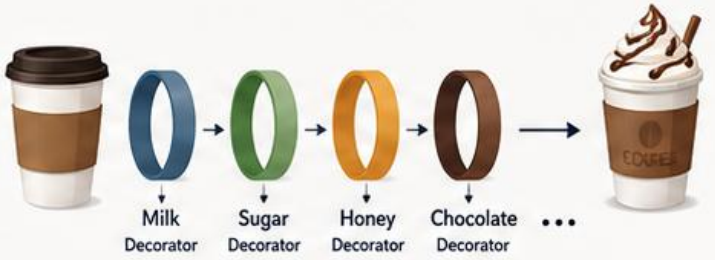
Every combination requires a new subclass.
Leads to class explosion!



VS

Using Decorator

Add responsibilities dynamically by wrapping objects.
No class explosion!



*The Decorator Pattern provides a **flexible** and **maintainable** way to extend object behavior at runtime.*

Real-World Examples

The Decorator Pattern lets you **add features** (responsibilities) to an object dynamically, without changing its core.

Coffee

Base Coffee

- **Milk** — Adds milk
- **Sugar** — Adds sugar
- **Caramel** — Adds caramel



Customized Coffee

Pizza

Base Pizza

- **Cheese** — Adds cheese
- **Mushrooms** — Adds mushrooms
- **Olives** — Adds olives



Customized Pizza

Car

Base Car

- **GPS** — Adds GPS navigation
- **Leather Seats** — Adds leather seats
- **Sunroof** — Adds sunroof



Customized Car

Email

Basic Email

- **Encryption** — Adds encryption
- **Compression** — Adds compression
- **Digital Signature** — Adds digital signature



Secured & Enhanced Email

Key Takeaway: Start with a simple object (base component) and decorate it with as many features as needed—**at runtime!**

Java Structure of the Decorator Pattern

The Decorator Pattern is built using **four key roles** arranged in this structure.

1. Component

- Common interface for all objects.
  Defines the operations that can be decorated.

2. Concrete Component

- The original object to be decorated.
  Provides the basic implementation of the Component interface.

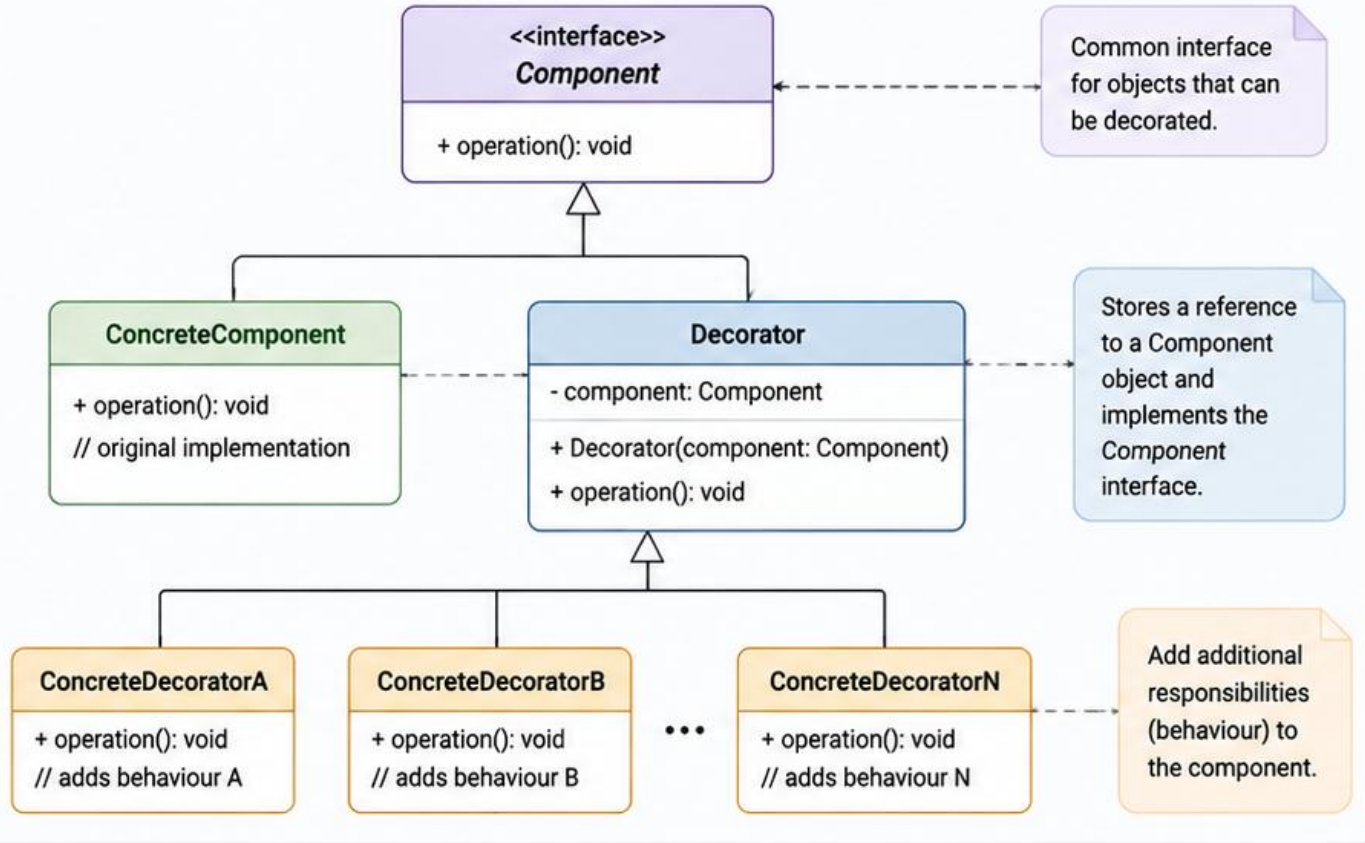
3. Decorator

- Also implements the Component interface.
- Stores a reference to a Component object.
  Acts as a wrapper and delegates requests to the wrapped component.

4. Concrete Decorators

- Extend the Decorator class.
- Add additional responsibilities (behaviour) before or after delegating the request.

UML Class Diagram



Example

```
Component coffee = new Espresso();          // Concrete Component
coffee = new MilkDecorator(coffee);         // Concrete Decorator
coffee = new SugarDecorator(coffee);        // Another Decorator
coffee.operation();                         // Result: Espresso + Milk + Sugar
```

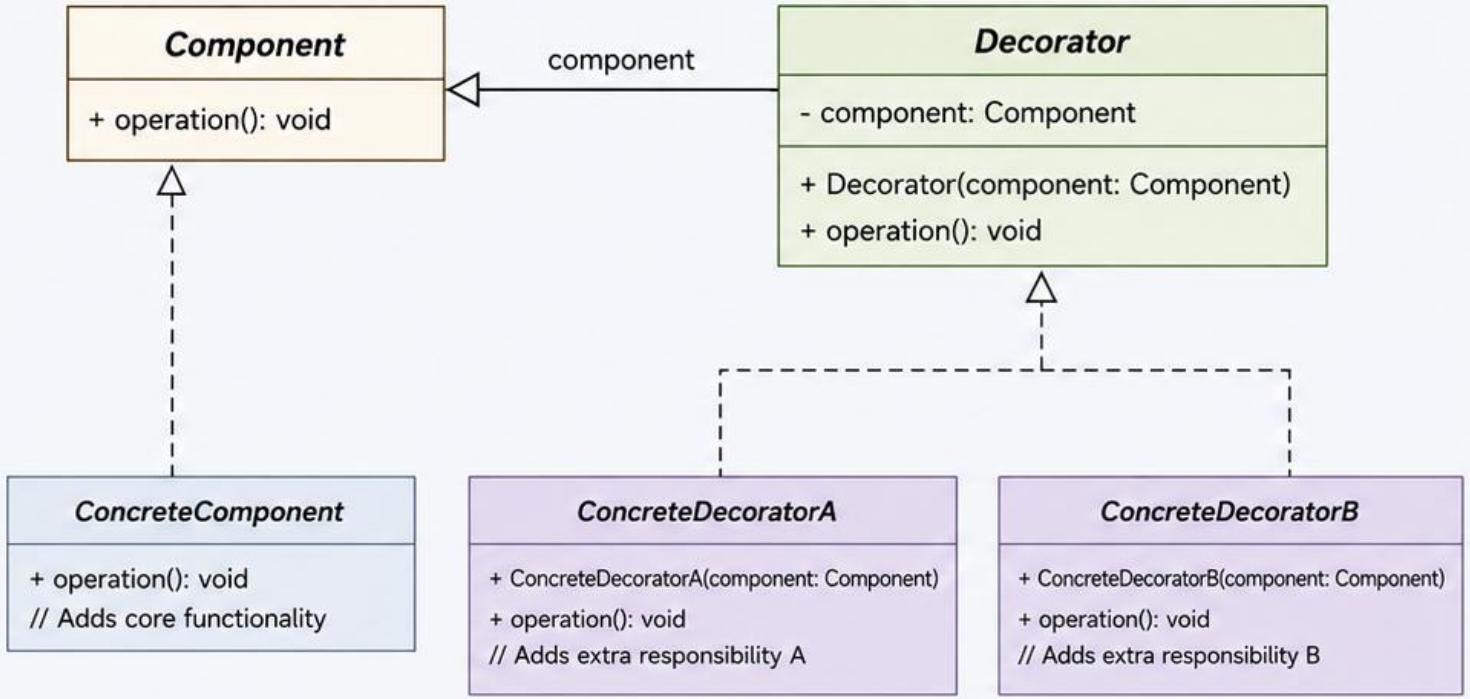
Key Takeaway

This structure allows you to add features **dynamically** and **flexibly** at runtime.

UML Diagram

Decorator Design Pattern

The Decorator Pattern attaches additional responsibilities to an object dynamically. Decorators provide a flexible alternative to subclassing for extending functionality.



Key Takeaway: Decorators wrap the component (composition) and add new responsibilities **dynamically**—without changing the original class.

What do these relationships mean?

extends (Generalization)
ConcreteComponent and ConcreteDecorator extend their parent classes to inherit structure and behavior.

implements (Realization)
Classes implement the Component interface (operations) so they can be used interchangeably.

has-a (Composition)
Decorator HAS-A Component. It wraps a Component object and forwards requests to it.

Question for Students

Which relationship is more important in the Decorator Pattern?

Answer: **Composition (has-a relationship)**

Java Implementation of Decorator Pattern

Example: Adding Milk and Sugar to Coffee Dynamically

1 Component

Interface

```
public interface Coffee {
    String getDescription();
    double getCost();
}
```

Defines the operations that can be **decorated**.

2 Concrete Component

```
public class Espresso implements Coffee {
    public String getDescription() {
        return "Espresso";
    }
    public double getCost() {
        return 350;
    }
}
```

The original object that provides the basic implementation.

3 Abstract Decorator

```
public abstract class CoffeeDecorator
        implements Coffee {

    protected Coffee coffee;

    public CoffeeDecorator(Coffee coffee) {
        this.coffee = coffee;
    }
}
```

Maintains a **reference** to a Coffee object and **implements** the Coffee interface.

4 Milk Decorator

```
public class MilkDecorator extends CoffeeDecorator {
    public MilkDecorator(Coffee coffee) {
        super(coffee);
    }
    public String getDescription() {
        return coffee.getDescription() + ", Milk";
    }
    public double getCost() {
        return coffee.getCost() + 80;
    }
}
```

Adds milk to the coffee.
Increases cost by **80**.

5 Sugar Decorator

```
public class SugarDecorator extends CoffeeDecorator {
    public SugarDecorator(Coffee coffee) {
        super(coffee);
    }
    public String getDescription() {
        return coffee.getDescription() + ", Sugar";
    }
    public double getCost() {
        return coffee.getCost() + 20;
    }
}
```

Adds sugar to the coffee.
Increases cost by **20**.

6 Client (Main)

```
public class Main {
    public static void main(String[] args) {
        Coffee coffee = new Espresso();          // base coffee
        coffee = new MilkDecorator(coffee);      // add milk
        coffee = new SugarDecorator(coffee);     // add sugar

        System.out.println(coffee.getDescription());
        System.out.println(coffee.getCost());
    }
}
```

Output



```
Espresso, Milk, Sugar    // Description
450.00                   // Total Cost
```

The Decorator Pattern allows us to add responsibilities (like Milk, Sugar) to objects dynamically by wrapping them with decorator objects, providing **flexible combinations**.

Espresso → + Milk → + Sugar

Suggested Java Package Structure

Example: Coffee Shop using Decorator Pattern

Project

- decorator/
  - Coffee.java
  - Espresso.java
  - CoffeeDecorator.java
  - MilkDecorator.java
  - SugarDecorator.java
  - ChocolateDecorator.java
  - CaramelDecorator.java
  - Main.java

Note:
All files are in the same package (decorator) for simplicity.

File / Class	Purpose
Coffee.java	Component interface defining operations (e.g., getDescription(), getCost()).
Espresso.java	ConcreteComponent – base coffee implementation.
CoffeeDecorator.java	Abstract Decorator class that implements Coffee and holds a reference to a Coffee object.
MilkDecorator.java	Concrete Decorator – adds milk.
SugarDecorator.java	Concrete Decorator – adds sugar.
ChocolateDecorator.java	Concrete Decorator – adds chocolate.
CaramelDecorator.java	Concrete Decorator – adds caramel.
Main.java	Client class to demonstrate the Decorator Pattern in action.

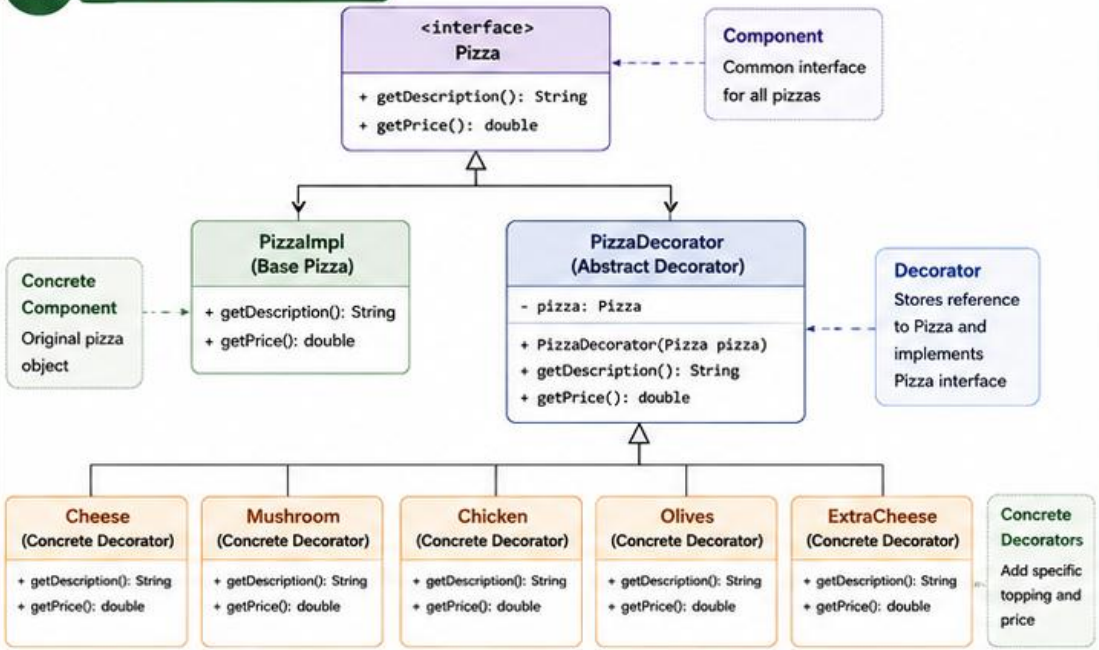
Decorator Pattern Example: Pizza Ordering System

Dynamically add toppings to a pizza using the Decorator Pattern

Scenario

A pizza shop allows customers to add toppings like Cheese, Chicken, and Olives to a base pizza dynamically.

UML CLASS DIAGRAM



JAVA IMPLEMENTATION

1) Component

```
public interface Pizza {
   String getDescription();
   double getPrice();
}
```

2) Concrete Component

```
public class PizzaImpl implements Pizza {
   public String getDescription() {
      return "Plain Pizza";
   }
   public double getPrice() {
      return 300.0;
   }
}
```

3) Abstract Decorator

```
public abstract class PizzaDecorator
         implements Pizza {
   protected Pizza pizza;

   public PizzaDecorator(Pizza pizza) {
      this.pizza = pizza;
   }
}
```

4) Cheese Decorator

```
public class Cheese extends PizzaDecorator {
  public Cheese(Pizza pizza) { super(pizza); }
  public String getDescription() {
     return pizza.getDescription() + ", Cheese";
  }
  public double getPrice() {
     return pizza.getPrice() + 60.0;
  }
}
```

5) Chicken Decorator

```
public class Chicken extends PizzaDecorator {
  public Chicken(Pizza pizza) { super(pizza); }
  public String getDescription() {
     return pizza.getDescription() + ", Chicken";
  }
  public double getPrice() {
     return pizza.getPrice() + 70.0;
  }
}
```

6) Olives Decorator

```
public class Olives extends PizzaDecorator {
  public Olives(Pizza pizza) { super(pizza); }
  public String getDescription() {
     return pizza.getDescription() + ", Olives";
  }
  public double getPrice() {
     return pizza.getPrice() + 40.0;
  }
}
```

CLIENT CODE (CREATE: CHEESE + CHICKEN + OLIVES)

```
public class PizzaOrderMain {
   public static void main(String[] args) {
      Pizza pizza = new PizzaImpl();          // Base pizza
      pizza = new Cheese(pizza);              // + Cheese
      pizza = new Chicken(pizza);             // + Chicken
      pizza = new Olives(pizza);              // + Olives

      System.out.println("Description : " + pizza.getDescription());
      System.out.printf("Total Price : %.2f\n", pizza.getPrice());
   }
}
```

Wrapping Order

The order of wrapping determines the final combination.

PizzaImpl → Cheese → Chicken → Olives

OUTPUT



```
Description : Plain Pizza, Cheese, Chicken, Olives
Total Price : 470.00
```

Breakdown

• Base Pizza	: 300.00
• Cheese	: 60.00
• Chicken	: 70.00
• Olives	: 40.00
Total	: 470.00

Successfully Decorated!

Toppings added dynamically at runtime.

Key Takeaway

The Decorator Pattern allows you to add toppings (responsibilities) to a pizza (object) dynamically without changing the base class, following the Open/Closed Principle.

Plain Pizza → + Cheese → + Chicken → + Olives

Abstract Class vs Interface

Two ways to achieve abstraction in Java

Abstract Class	Interface
Common base class	Common capability
IS-A	CAN-DO
Shared attributes	Behaviour contract
extends	implements

Remember:

Use **Abstract Class** when classes are related (**IS-A**).

Use **Interface** when classes are unrelated but share a common capability (**CAN-DO**).

Decorator vs Inheritance

Two different ways to add responsibilities to a coffee

Inheritance

Creates a **new class** for every combination



- **Coffee** — getCost()
- **MilkCoffee** — getCost()
- **SugarMilkCoffee** — getCost()
- **ChocolateSugarMilkCoffee** — getCost()

Each new responsibility requires a **new subclass.**

⚠ Leads to **many classes** and **deep inheritance** hierarchies

Decorator

Wraps the object with decorators



- **Coffee** — getCost()
- **MilkDecorator** — getCost() + milkCost
- **SugarDecorator** — getCost() + sugarCost
- **ChocolateDecorator** — getCost() + chocolateCost

Add or remove responsibilities by wrapping **without creating new classes.**

✓ Flexible, simple, and **avoids class explosion**

Key Difference

Inheritance creates classes.

Every combination results in a new subclass at compile time.

Decorator creates objects.

Responsibilities are added dynamically at runtime by wrapping the object.

Remember

Use Decorator when you want to add responsibilities **dynamically** and keep your design **flexible.**

Key Takeaway: Inheritance leads to **class proliferation** and **rigid hierarchies.** Decorator provides a **flexible alternative** by composing objects.



Prefer **Composition (Decorator)** over deep **Inheritance.**

Decorator vs Proxy

Both are structural patterns, but solve different problems.

DECORATOR

Adds responsibilities to an object dynamically

Purpose	Add **functionality** to an object without changing its structure.
How it works	Wraps the original object with one or more decorator objects.
Focus	Extends behavior / adds new responsibilities.

Examples: Encryption, Compression, Logging, Milk, Sugar

Decorator **extends** what the object can do.
It **enhances** the object.

VS

PROXY

Controls access to the original object

Purpose	Control **access** to an object for security, performance, or other reasons.
How it works	Acts as a surrogate or placeholder for the real object.
Focus	Manages access / controls when and how the object is used.

Examples: Authentication, Lazy Loading, Security, Remote Object

Proxy **controls** how and when the object is accessed.
It **protects** or **manages** the object.

Key Differences

- Decorator adds **responsibilities**. Proxy controls **access**.
- Decorator can be **stacked** multiple times. Proxy is usually **one layer** of control.
- Decorator changes **behavior**. **Proxy** governs **usage**.
- Decorator is about **enhancement**. Proxy is about **protection**.

Remember

Use Decorator when you need to **add features** dynamically.

Use **Proxy** when you need to **control access** to an object.

Decorator Pattern: Advantages & Disadvantages

Advantages

Runtime flexibility	Add or remove responsibilities to objects at runtime.
No class explosion	Avoids large number of subclasses for every possible combination.
Follows Open/Closed Principle	Open for extension, closed for modification.
Reusable decorators	Decorators can be reused to add behavior to different objects.
Small focused classes	Each class has a single responsibility and is easier to understand and maintain.

Disadvantages

Many small classes	Creates many decorator classes, which can increase complexity.
Harder debugging	Multiple layers of wrapping make debugging more difficult.
Wrapping order matters	The order in which decorators are applied affects the final behavior and result.
More objects created	Each decorator adds another object to the structure, increasing memory usage and object management.

The Decorator Pattern provides a **flexible** and **maintainable** way to extend object behavior dynamically.

Use the Decorator Pattern when you need **flexible extension**, but be aware of **added complexity**.

When to Use:

- When you need to add responsibilities to objects dynamically.
- When inheritance would lead to a large number of classes.

When Not to Use:

- When the core behavior changes frequently.
- When added responsibilities are not optional.



Summary

What We Learned Today

Today we learned

Decorator Pattern	A structural design pattern that allows adding new responsibilities to objects dynamically.
Runtime Behaviour Extension	Responsibilities can be added or removed at runtime without changing the class.
Composition over Inheritance	Uses composition to achieve flexibility and avoid class explosion.
Java Implementation	Implemented the pattern in Java using interfaces, abstract classes and decorators.
UML Structure	Understood the roles: Component, Concrete Component, Decorator and Concrete Decorators.
Decorator vs Proxy	Explored the key differences in purpose and usage between Decorator and Proxy patterns.
Real-world Applications	Applied the pattern in examples like Coffee Shop and Pizza Ordering System.

Key Takeaway

The Decorator Pattern helps us extend object behaviour **flexibly** at runtime while keeping our code **open** for extension and **closed** for modification.

Decorator Pattern in Action



Base Coffee	+ Milk	+ Sugar	+ Chocolate	= Custom Coffee
Espresso	MilkDecorator	SugarDecorator	ChocolateDecorator	Extended at Runtime

Just like toppings on coffee or pizza, decorators add value, one by one, as needed!



Great job! You are now one step closer to becoming a better designer and developer.